Q : 1 Understanding how generators work in Python.

Ans : A generator in Python is a special type of function that allows you to iterate over a sequence of values without storing the entire sequence in memory. It is defined using the yield keyword instead of return.

When a generator function is called, it does not execute immediately. Instead, it returns a generator object. The execution starts only when the generator is iterated using functions like next() or a loop.

Q : 2 Difference between yield and return.

Ans :

Feature	return	yield
Function Type	Normal function	Generator function
Execution	Ends function immediately	Pauses function and resume later
Output	Returns a single value	Produces a sequence of values
State Retention	Does not retain state	Retains state between calls
Memory Usage	Stores full result in memory	Generates values one at a time

Q : 3 Understanding iterators and creating custom iterators.

Ans : An iterator is an object that allows traversal through all the elements of a collection (such as lists, tuples, etc.). It implements two methods:

```
numbers = [1, 2, 3]
it = iter(numbers)

print(next(it))
print(next(it))
```

Q : 4 How functional programming works in Python

Ans : Functional programming is a programming paradigm where computation is treated as the evaluation of mathematical functions. It focuses on writing pure functions, avoiding changing data (immutability), and using functions as first-class objects.

Q : 5 Using map(), reduce(), and filter() functions for processing data.

Ans :

(a) `map()` Function

- The `map()` function applies a given function to each item of an iterable and returns a map object (iterator).

```
numbers = [1, 2, 3, 4]

squares = list(map(lambda x: x * x, numbers))
print(squares)
```

(b) filter() Function

The filter() function filters elements from an iterable based on a condition (function returning True/False).

```
numbers = [1, 2, 3, 4, 5]
even_numbers = list(filter(lambda x: x % 2 == 0, numbers))
print(even_numbers)
```

(c) reduce() Function

The reduce() function applies a function cumulatively to items of an iterable to reduce it to a single value. It is available in the functools module.

```
from functools import reduce
numbers = [1, 2, 3, 4]
sum_all = reduce(lambda x, y: x + y, numbers)
print(sum_all)
```

Q : 6 Understanding how to create and access elements in a list. Indexing in lists (positive and negative indexing). Slicing a list: accessing a range of elements.

Ans : A list in Python is an ordered and mutable collection of elements. It can store multiple items of different data types such as integers, strings, or even other lists.

```
l = list((1,2,3,4,5,6,7,8,9))

print(l)

print(len(l))

print(type(l))


print(l[0])

print(l[-1])

print(l[2:])

print(l[2:5])
```

```
print(l[-4:-1])
print(l[1:8:2])
print(l[::-1])
```

Q : 7 Common list operations: concatenation, repetition, membership.

Ans : 1. Concatenation of Lists

Concatenation means joining two or more lists into a single list. This is done using the + operator.

Example:

```
list1 = [1, 2, 3]
list2 = [4, 5, 6]

result = list1 + list2
print(result)
```

2. Repetition of Lists

Repetition means repeating the elements of a list multiple times. This is done using the * operator.

Example:

```
numbers = [1, 2, 3]

result = numbers * 3
print(result)
```

3. Membership in Lists

Membership is used to check whether an element exists in a list or not. This is done using the in and not in operators.

Example:

```
numbers = [10, 20, 30, 40]

print(20 in numbers)
print(50 in numbers)
print(50 not in numbers)
```

Q : 8 Understanding list methods like append(), insert(), remove(), pop().

Ans : 1. append() Method

The append() method is used to add an element at the end of the list.

Syntax:

list.append(element)

Example:

```
numbers = [1, 2, 3]
numbers.append(4)

print(numbers)
```

2. insert() Method

The insert() method is used to add an element at a specific position (index).

Syntax:

list.insert(index, element)

Example:

```
numbers = [1, 2, 3]
numbers.insert(1, 10)
print(numbers)
```

3. remove() Method

The remove() method is used to remove the first occurrence of a specific element from the list.

Syntax:

list.remove(element)

Example:

```
numbers = [1, 2, 3, 2]
numbers.remove(2)

print(numbers)
```

4. pop() Method

The pop() method is used to remove and return an element from the list. By default, it removes the last element, but you can also specify an index.

Syntax:

list.pop([index])

Example:

```
numbers = [1, 2, 3, 4]

numbers.pop()      # removes last element
numbers.pop(1)     # removes element at index 1

print(numbers)
```

Q : 9 Iterating over a list using loops.

Ans :

```
numbers = [10, 20, 30, 40]

    for num in numbers:
    print(num)
```

Q : 10 Sorting and reversing a list using sort(), sorted(), and reverse().

Ans :

a) sort() Method

- Sorts the list in-place (modifies original list)

```
numbers = [4, 2, 1, 3]
numbers.sort()

print(numbers)  # [1, 2, 3, 4]
```

For descending order:

```
numbers.sort(reverse=True)
```

(b) sorted() Function

- Returns a new sorted list without modifying the original

```
numbers = [4, 2, 1, 3]

new_list = sorted(numbers)
print(new_list)   # [1, 2, 3, 4]
```

(c) reverse() Method

- Reverses the order of elements in the list

```
numbers = [1, 2, 3, 4]
numbers.reverse()
```

```
print(numbers)  # [4, 3, 2, 1]
```

Q : 11 Basic list manipulations: addition, deletion, updating, and slicing.

Ans :

(a) Addition of Elements

```
numbers = [1, 2, 3]

numbers.append(4)
numbers.insert(1, 10)
```

(b) Deletion of Elements

```
numbers = [1, 2, 3, 4]

numbers.remove(2)
numbers.pop()
del numbers[0]
```

(c) Updating Elements

```
numbers = [1, 2, 3]

numbers[1] = 20
print(numbers)
```

(d) Slicing

```
numbers = [10, 20, 30, 40, 50]

print(numbers[1:4])
```

Q : 12 Introduction to tuples, immutability.

Ans : A tuple in Python is an ordered collection of elements, similar to a list. However, the key difference is that tuples are immutable, meaning their elements cannot be changed after creation.

Q : 13 Creating and accessing elements in a tuple.

Ans :

```
my_tuple = (1, 2, 3)
my_tuple[1] = 10
```

Q : Basic operations with tuples: concatenation, repetition, membership.

Ans : (a) Concatenation

```
t1 = (1,2)
t2=(3,4)
result =t1+t2
print(result)
```

(b) Repetition

```
t = (1, 2)
result = t * 3
print(result)
```

(c) Membership

```
t = (10, 20, 30)
print(20 in t)      # True
print(40 not in t)  # True
```

Q : 15 Accessing tuple elements using positive and negative indexing.

Slicing a tuple to access ranges of elements.

Ans :

```
t = (10,32,60,59,45,"meet",'A',45.66,True)
print(t[0])
print(t[2:5])
print(t[-5])
print(t[::-1])
```

Q : 16 Introduction to dictionaries: key-value pairs.

- Accessing, adding, updating, and deleting dictionary elements.
- Dictionary methods like keys(), values(), and items().

Ans : A dictionary in Python is an unordered collection of data stored in key-value pairs. Each key is unique and is used to access its corresponding value.

(a) Accessing Elements

You can access values using their keys.

```
student = {"name": "John", "age": 20}

print(student["name"])
```

(b) Adding Elements

You can add new key-value pairs by assigning a value to a new key.

```
student["grade"] = "A"
```

(c) Updating Elements

You can update an existing value by assigning a new value to the key.

```
student["age"] = 21
```

(d) Deleting Elements

You can remove elements using del or pop().

```
del student["course"]
student.pop("age")
```

(a) keys() Method

Returns all the keys in the dictionary.

```
student = {"name": "John", "age": 20}

print(student.keys())
```

(b) values() Method

Returns all the values in the dictionary.

```
print(student.values())
```

(c) items() Method

Returns all key-value pairs as tuples.

```
print(student.items())
```

Q : 17 • Iterating over a dictionary using loops.

• Merging two lists into a dictionary using loops or zip().

• Counting occurrences of characters in a string using dictionaries.

Ans :

(a) Iterating Through Keys

```
student = {"name": "John", "age": 20, "course": "Python"}
for key in student:
    print(key)
```

(b) Iterating Through Values

```
for value in student.values():
    print(value)
```

(c) Iterating Through Key-Value Pairs

```
for key, value in student.items():
     print(key, value)
```

```
*).  keys = ["name", "age", "course"]
     values = ["John", 20, "Python"]

     result = dict(zip(keys, values))
     print(result)
```

```
*). text = "hello"

    char_count = {}

    for char in text:
        if char in char_count:
           char_count[char] += 1
        else:
           char_count[char] = 1

    print(char_count)
```

Q : 18 • Defining functions in Python.

• Different types of functions: with/without parameters, with/without return values.

• Anonymous functions (lambda functions).

Ans : A function in Python is a block of organized and reusable code used to perform a specific task. Functions help in improving code readability, reusability, and modularity.

- ```
  def greet():
      print("Hello")

  greet()
  ```
- ```
  def greet(name):
      print("Hello", name)

  greet("Alice")
  ```
- ```
  def get_number():
      return 10

  num = get_number()
  print(num)
  ```

- **Lambda Function :**
  ```
  add = lambda a, b: a + b

  print(add(2, 3))
  ```

Q : 19 • Introduction to Python modules and importing modules.

• Standard library modules: math, random.

• Creating custom modules.

Ans : A module in Python is a file that contains Python code such as functions, variables, and classes. Modules help organize code into separate files, making programs more structured and reusable.

- ```
  import math

  print(math.sqrt(16))
  print(math.pow(2, 3))
  ```
- ```
  import random

  print(random.randint(1, 10))
  print(random.choice([1, 2, 3]))
  ```

- **Custom Module**
  ```
  def greet(name):
      return "Hello " + name
  ```
  ```

```
def add(a, b):
    return a + b

import mymodule

print(mymodule.greet("Alice"))
print(mymodule.add(2, 3))
```